atmosphere

MDPI

Article

A Novel Komodo Mlipir Algorithm and Its Application in PM2.5 Detection

Linxuan Li and Ming Zhao *

School of Computer Science, Yangtze University, Jingzhou 434023, China
* Correspondence: hitmzhao@gmail.com

Abstract: The paper presents an improved Komodo Mlipir Algorithm (KMA) with variable inertia weight and chaos mapping (VWCKMA). In contrast to the original Komodo Mlipir Algorithm (KMA), the chaotic sequence initialization population generated by Tent mapping and Tent Chaos disturbance used in VWCKMA can effectively prevent the algorithm from falling into a local optimal solution and enhance population diversity. Individuals of different social classes can be controlled by the variable inertia weight, and the convergence speed and accuracy can be increased. For the purpose of evaluating the performance of the VWCKMA, function optimization and actual predictive optimization experiments are conducted. As a result of the simulation results, the convergence accuracy and convergence speed of the VWCKMA have been considerably enhanced for single-peak, multi-peak, and fixed-dimensional complex functions in different dimensions and even thousands of dimensions. To address the nonlinearity of PM2.5 prediction in practical problems, the weights and thresholds of the BP neural network were iteratively optimized using VWCKMA, and the BP neural network was then used to predict PM2.5 using the optimal parameters. Experimental results indicate that the accuracy of the VWCKMA-optimized BP neural network model is 85.085%, which is 19.85% higher than that of the BP neural network, indicating that the VWCKMA has a certain practical application.

Keywords: Komodo Mlipir Algorithm; Tent mapping; inertia weight; chaos search; PM2.5 prediction

check for updates

Citation: Li, L.; Zhao, M. A Novel Komodo Mlipir Algorithm and Its Application in PM2.5 Detection. *Atmosphere* **2022**, *13*, 2051. https://doi.org/10.3390/atmos13122051

Academic Editors: HwaMin Lee and Sungroul Kim

Received: 12 November 2022
Accepted: 5 December 2022
Published: 7 December 2022

Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The metaheuristic algorithm is a type of optimization method that is based on biological cluster behavior or natural phenomenon rules. Since metaheuristic algorithms possess the characteristics of flexibility, simplicity, and efficiency, they have been widely used to solve problems such as the traveling salesman problem [1,2], the robot path planning problem [3,4], the workshop scheduling problem [5,6], and a wide range of other complex optimization problems with remarkable effects. The concept of intelligent optimization algorithms has been widely known since the mid to late 20th century. The Particle Swarm Optimization [7,8] (PSO) algorithm, for example, simulates the foraging behavior of birds, the Ant Colony Optimization [9] (ACO) algorithm simulates the foraging behavior of ants, while natural selection in evolutionary theory is the source of inspiration for the Genetic Algorithm [10,11] (GA). The rapid development of information technology has led to the development of more and more intelligent optimization algorithms to meet actual needs. Relevant scholars have proposed the following algorithms, including the Sparrow Search Algorithm [12] (SSA) based on sparrow foraging behavior and anti-predation behavior, the Grey Wolf Optimizer [13] (GWO) algorithm, which simulates the rank and predation mechanism of the grey wolf, the Whale Optimization Algorithm [14] (WOA) based on the predation behavior of the humpback whale, the Dwarf Mongoose Optimization [15] (DMO) algorithm, which mimics the foraging behavior of the dwarf mongoose, the Slime Mould Algorithm [16] (SMA) based on the oscillation mode of slime mould, the Barnacle Mating Optimizer [17] (BMO) algorithm, which mimics the mating behavior of barnacles,

the Galactic Swarm Optimization [18] (GSO) algorithm, which is inspired by the motion of stars within galaxies, the Search And Rescue [19] (SAR) algorithm, which imitates the exploration behavior of humans during a search, and the Komodo Mlipir Algorithm [20] (KMA), which simulates Komodo dragon foraging and mating behavior, etc. Furthermore, metaheuristic algorithms that combine deep learning and neural networks for hyperparameter tuning have always been a hot topic in research. For example, CKH (Chaotic Krill Herd) algorithms are proposed to select the optimal parameters of the following algorithms, such as support vector regression [21] (SVR), BP neural network based on improved particle swarm optimization [22], and twin support vector machine based on improved artificial fish swarm algorithms [23]. It is also possible to optimize the parameters of deep learning models, such as convolutional neural network optimized based on artificial fish swarm optimization [24] and stacking network, which is optimized by particle swarm optimization algorithm [25].

The Komodo Mlipir Algorithm (KMA) is a novel metaheuristic optimization algorithm that was proposed by Suyanto et al. [20] in January 2022. The design is based on the behavior of Komodo monitor lizards located on Komodo Island and the Lesser Sunda Islands near Flores and the Mlipir motion. The KMA is based on swarm intelligence inspired by the social hierarchy, foraging, and reproduction of the Komodo dragon in nature. In small males, KMA used Mlipir motion as a higher exploration strategy, combining the characteristics of the Genetic Algorithm and Particle Swarm Optimization algorithm. Depending on the quality of the large males, females can choose to reproduce sexually (exploitation) or asexually (exploration). Males of large size can choose to attract one another (exploitation) or repel one another (exploration). As a final point, self-adaptation is proposed as a means of controlling the balance between exploitation and exploration and increasing the diversity of species. KMA can balance exploration and exploitation effectively, can have faster convergence speed, and can accurately converge to the optimal value by using adaptive population. However, KMA still has the problem of local convergence in the solution of complex functions, such as converging to a local optimum in the Rastrigin function. In the in-depth discussion of the Komodo Mlipir Algorithm, it is discovered that the importance of large males, small males, and females in the KMA is the same, but in the normal social hierarchy, the three classes should have different levels of importance. Consequently, the algorithm occasionally falls into the local optimal solution, and the convergence result deviates from the global optimal.

An improved Komodo Mlipir Algorithm with variable inertia weight and chaos map (VWCKMA) is proposed in this paper to further enhance the performance of KMA. In terms of higher-dimensional reference functions, VWCKMA has been demonstrated to be stable. There are three main contributions of the study. (1) By introducing time-varying inertia weights, the social class of each Komodo dragon is given an inertia weight, which increases the convergence rate of the population and enhances global exploration ability. (2) A Tent chaotic mapping approach is employed to initialize the Komodo dragon population in order to enhance the quality of the initialized population position. (3) The Tent chaotic model is employed in order to generate a few local solutions and compare them with the current solution. The algorithm will dynamically escape the local optimum if the local solution is better than the current one. As a result, the population will be more diverse, and the VWCKMA will be able to adjust the diversity of the population.

Section 3 introduces the standard KMA, and Section 4 presents the Tent mapping initialization population, the variable weight governing equation, and the Tent chaos perturbation. Section 5 discusses the experimental setup and comparison of functional optimization experiments. Section 6 presents the practical application of optimized BP neural networks for the prediction of PM2.5. Section 6 summarizes the findings and suggests future research directions.

2. Basic Komodo Mlipir Algorithm

The Komodo dragon is a leftover species from the age of dinosaurs, belonging to the family of Varanidae, living on the Komodo Islands and the Lesser Sunda Islands near Flores. Komodo dragons are ferocious, and males occasionally prey on weaker siblings and larvae and can even forage for food several kilometers away. Male monitor lizards fight to decide who will mate. Komodo dragon females are occasionally parthenogenetic; that is, they do not need males to produce offspring.

In the Komodo Mlipir Algorithm, the social hierarchy, foraging, and reproduction of the Komodo dragon are simulated. n Komodo dragon individuals were classified as large males, females, and small males based on their fitness (qualities). In the KMA, large and small males are optimized by their movements, and females are optimized by their reproduction. A concept of Mlipir is introduced in the Komodo Mlipir Algorithm. In small groups of males, in addition to foraging for themselves, they will commonly hunt for leftovers from large male Komodo dragon individuals to feed on. However, this behavior carries the risk of predation by large males, so they evade this hazard to feed safely. We call this behavior Mlipir, the movement of small males.

First, low-quality large males are attracted by high-quality large males, while high-quality large males are attracted or rejected by low-quality large males with a 0.5 probability of gaining a new position. According to the mathematical model, large males behave as follows:

$$V_{ij} = \begin{cases} r_1(k_j - k_i) \text{ if } f(k_j) < f(k_i) \text{ or } r_2 < 0.5 \\ r_1(k_i - k_j) \text{ otherwise} \end{cases} \tag{1}$$

$$k_i' = k_i + \sum_{j}^{q} V_{ij}, \text{where } j \neq i, \tag{2}$$

where $f(k_j)$ and $f(k_i)$ are represent the fitness (or quality) of the j and i large males, respectively. k_j and k_i represent the positions ($j \neq i$) of the j and i large males, r_1 and r_2 represent the random numbers in the interval [0, 1] in the normal distribution, respectively, and q represents the number of large males.

Second, the exploitation by mating a female with a large male of the highest quality to produce two offspring, or the exploration by doing parthenogenesis. Mathematically, the mating process (sexual reproduction) of two individuals can be described as follows:

$$\begin{array}{l} k_{il}' = r_l \cdot k_{il} + (1 - r_l) \cdot k_{jl} \\ k_{jl}' = r_l \cdot k_{jl} + (1 - r_l) \cdot k_{il} \end{array} \tag{3}$$

where k_{il} and k_{jl} represent the highest quality large male and female in the l dimension, k_{il}' and k_{jl}' represent the two offspring of the l dimension produced by the mating process of the highest quality large male and female, and r_l represents the random numbers in the interval of the l dimension normal distribution [0, 1].

In order to achieve parthenogenesis procedure (asexual reproduction), a small value is assigned to each female Komodo dragon's individual dimension. A small value is generated randomly using a symmetric normal distribution and is mathematically defined as follows:

$$(k_{i1}, k_{i2}, \cdots, k_{im}) \rightarrow (k_{i1}', k_{i2}', \cdots, k_{im}') \tag{4}$$

$$k_{ij}' = k_{ij} + (2r - 1)\alpha \left| ub_j - lb_j \right| \tag{5}$$

where $k_{i1}, k_{i2}, \ldots, k_{im} \in \left[lb_j, ub_j \right]$ represents the position vector of k Komodo dragon individual in the m dimension, lb_j and ub_j represent the lower bound and upper bound of the j dimension, respectively, r represents a random value in normal distribution and α represents a parthenogenesis radius, and is set to a fixed value of 0.1. As a result, new solutions can be generated within 10% of the search space radius.

Third, in mathematical terms, the movement of small males by randomly selecting a portion of them to approach the large male can be described as follows:

$$V_{ij} = \begin{cases} \sum_{l=1}^{m} r_1(k_{jl} - k_{il}) & if\ r_2 < d \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

$$k_i' = k_i + \sum_{q}^{j=1} V_{ij}, \text{ where } j \neq i, \tag{7}$$

where r_1 and r_2 represent the random numbers in the interval [0, 1] in the normal distribution, k_{il} and k_{jl} represent the i small male and j large male in the l dimension, d represents the mlipir probability, l represents the dimension randomly selected based on the normal distribution, q represents the number of large males.

To summarize, the optimization process of the KMA can be described as follows: randomly generate Komodo dragon populations and determine three different classes of large males, females, and small males. The final optimization process is accomplished by moving the three classes in different directions.

3. Improvements of Basic Komodo Mlipir Algorithm

3.1. Proposed Initialization of Position by Chaotic Sequence

Chaotic mapping is a random and irregular motion. Random initialization of particles is used in most algorithms, but this may lead to an uneven distribution of particles. A chaotic sequence is characterized by ergodicity and inherent randomness in space; it can repeat irregular ergodicities within a certain range, jump out of local optimal solutions, and achieve global optimization. In recent studies, it has been demonstrated that the optimization efficiency of a swarm intelligence algorithm optimization can be greater than that of a random algorithm when chaos theory is applied, and the Tent Chaos Model has a higher ergodic property compared to the Logistic Chaos Model [26]. As a result, this paper utilizes the Tent chaotic mapping model.

Tent mapping has a simple structure, and its iterative process makes it suitable for running on a computer. There is a greater degree of ergodic uniformity in it. In this paper, the population is initialized using a Tent chaotic mapping model. Equation (8) expresses the Tent chaotic mapping model mathematically as follows:

$$\alpha_{ij}^{t+1} = \begin{cases} 2\alpha_{ij}^{t} & \alpha_{ij}^{t} \in (0, 0.5] \\ 2(1 - \alpha_{ij}^{t}) & \alpha_{ij}^{t} \in (0.5, 1] \end{cases} \tag{8}$$

where t represents the number of iterations, $i = 1, 2, \cdots, n$. n represents the population size, $j = 1, 2, \cdots, d$, and d represents the dimension size. α_{ij}^{t} represents a chaotic variable on the j dimensional component of the i particle.

The Tent mapping method is used to generate a randomly distributed initial population to guarantee the randomness of individuals in the initial population. As a result, the search area for individual Komodo dragons is broadened and the diversity of group locations is reinforced.

3.2. Proposed Variable Weight Strategy

The importance of large males, small males, and females in the KMA is the same, but in the normal social hierarchy, the three classes should have different levels of importance. Thus, the KMA occasionally falls into the local optimal solution. For the purpose of preventing the KMA from falling into the local optimal value during search, a new parameter called inertia weight is introduced into the original Komodo Mlipir Algorithm. In this paper, this new parameter controls the speed and individual optimum, thus affecting both global exploration and local search. When it comes to the motion of large males, females,

and small males, different inertial weights, $w1$, $w2$ and $w3$ are introduced, respectively. The first step is to vary all the weights and add them up to 1.0.

$$w1 + w2 + w3 = 1 \tag{9}$$

For large males, females, and small males, the inertia weight should always be $w1 \geq w2 \geq w3$. During the initial iteration of determining the global optimal value, the global exploration has a greater influence. As the number of iterations increases, the influence of local optimization increases at the later stage of the process, so the inertia weight should change as well. During the iteration process, the weight of large males should be reduced from 1.0 to 1/3. Meanwhile, female reproduction and the movement of small males following large males are examples of local search. The weight of and will be increased from 0.0 to 1/3. A control inertia weight equation is presented in this paper. The cosine function is used to describe $w1$, and the angle θ is limited between $[0, \arccos(1/3)]$. For $w2$, when the number of iterations is 0, $w2$ and $w3$ approach 0. When the number of iterations approaches ∞, $w1$, $w2$ and $w3$ weight approaches 1/3. Thus, the parameter is introduced, which is expressed mathematically as follows:

$$\varphi = \frac{1}{2}\arctan(iter) \tag{10}$$

Due to the fact that the limiting angle θ falls between $[0, \arccos(1/3)]$ and $w2$ will approach from 0 to 1/3 as the number of iterations increases. It is assumed that $w2$ contains $\sin\theta$ and conφ, and θ should be from $0 \rightarrow \arccos(1/3)$. The mathematical expression is as follows:

$$\theta = \frac{2}{\pi}\arccos\frac{1}{3} \cdot \arctan(iter) \tag{11}$$

There is a new variable weight location update method proposed, which is defined as follows:

$$\begin{aligned} w1 &= \cos\theta, \\ w2 &= \tfrac{1}{2}\sin\theta \cdot \cos\varphi, \\ w3 &= 1 - w1 - w2, \end{aligned} \tag{12}$$

The varying weight curve is illustrated in Figure 1.

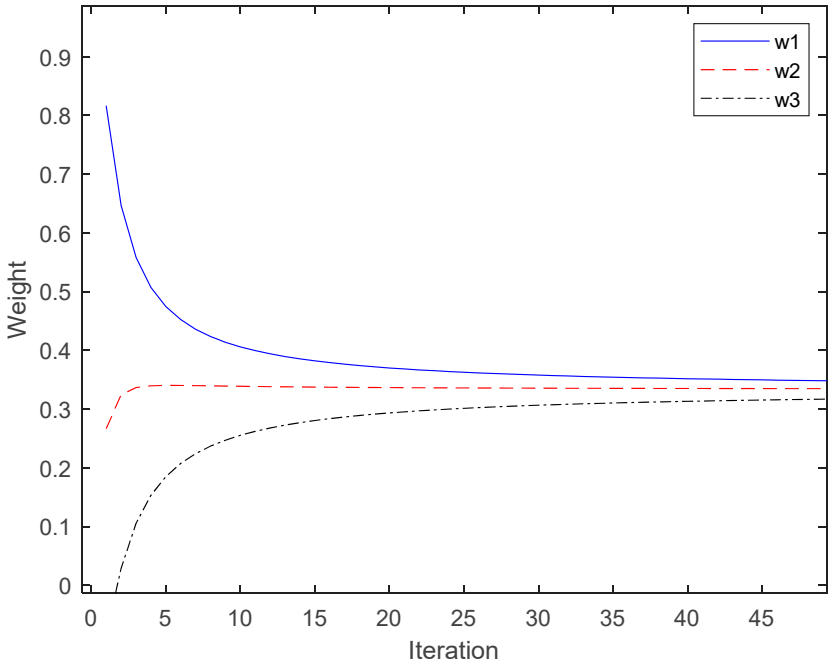


Figure 1. Iterative graph of time-varying weights.

As a result, there has been an improvement in the mathematical expression (1) of large male movement as follows:

$$V_{ij} = \begin{cases} w1 * V_{ij} + r_1(k_j - k_i) \text{ if } f(k_j) < f(k_i) \text{ or } r_2 < 0.5 \\ w1 * V_{ij} + r_1(k_i - k_j) \text{ otherwise} \end{cases} \quad (13)$$

where k_i and k_j represent different large males, r_1 and r_2 represent a random number between [0, 1], and $w1$ represents the inertia weight of large males.

According to the original KMA, females can only reproduce sexually or asexually. The new movement of the female toward the large male is in accordance with the normal physiological activity in nature. In essence, this movement is intended to move toward the direction of large males for local exploration in order to determine the optimal value. This movement can be expressed mathematically as follows:

$$\begin{array}{l} V_i = w2 * V_i + r * \left(k_j - k_i\right), \text{where } j \neq i, \\ k_i' = k_i + V_i \end{array} \quad (14)$$

where k_i and k_j represent females and the largest males, respectively; r represents a random number between [0, 1], and $w2$ represents the inertia weight of females. k_i' represents the new female position.

In a similar manner, the movement of small males is improved by adding inertia weight, i.e., Equation (6) is improved. Therefore, the mathematical expression for the position movement of small males is as follows:

$$V_{ij} = \begin{cases} w3 * V_{ij} + \sum_{l=1}^{m} r_1(k_{jl} - k_{il}) & if\ r_2 < d \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

where r_1 and r_2 represent the random numbers in the interval [0, 1] in the normal distribution; k_{il} and k_{jl} represent the i small male and the j large male in the l dimension, respectively; d represents the specific probability; l represents the dimensions randomly selected based on the normal distribution, and $w3$ represents the inertia weight of small males.

3.3. Proposed Tent Chaos Disturbance Strategy

After the movement of the three classes of Komodo is completed, that is, in the case of global exploration, chaotic disturbance is executed to achieve local search. This paper uses the Tent chaotic model for perturbation to generate some local solutions around the Komodo volume, which are then compared with preceding solutions. The specific Tent chaotic local search steps are as follows:

Step 1: Set $k = 0$, randomly generate a random number m of [0, Dim], which means that chaotic search is conducted on dimension m, and randomly generate a random variable α_{ij}^t of [0, 1].

Step 2: Via Equation (8) to generate chaotic variable α_{ij}^{t+1}, generate a local solution around the ith individual in the jth dimension: $x_{ij}^{t+1} = \alpha_{ij}^{t+1} \times x_{ij}^t \ j = 1, 2, \cdots, m$.

Step 3: Determine whether x_{ij}^{t+1} meets the constraint conditions and standardize them within the specified range.

Step 4: Calculated fitness of x_{ij}^{t+1}. In the event that the local solution is superior to the original solution, the obtained local solution will replace the original particle, and if a solution superior to the original particle cannot be generated after reaching the maximum chaotic time, the local solution will be discarded; otherwise, $k = k + 1$, return to step 2.

3.4. VWCKMA Process

Step 1: Initialize the parameters. The size of the population and dimension, the maximum and minimum iterations, radius of parthenogenesis, the probability value of the mlipir, and the maximum amount of chaos.

Step 2: Initialize the population. Komodo dragon individual groups are initially positioned using Tent mapping.

Step 3: Calculate the fitness values of Komodo dragon individuals and the inertia weights of Komodo dragon of different social classes by using Equations (9)–(12).

Step 4: Update the position of large males according to Equations (2) and (13). Then, update the position of females based on Equations (3)–(5) and (14), as well as the position of small males based on Equations (7) and (15).

Step 5: Use Equation (8) to conduct a local search and generate a local solution: $x'_{il} = \alpha^t_{ij} \times x_{ij}$, where $i = 1, 2, \cdots, n$, n represents the population size, $j = 1, 2, \cdots, d$, and d represents the dimension size. Determine whether x'_{il} meets the constraint conditions, and limit the solution to the variable range.

Step 6: Calculated fitness value of x'_{il}. If the obtained local solution is superior to the original solution, the obtained local solution replaces the original particle, and if a solution superior to the original particle cannot be generated after reaching the maximum chaotic times, the local solution is discarded. Otherwise, return to Step 5.

Step 7: Execute Step 8 after determining that the optimal solution is obtained or that the maximum number of iterations is reached; otherwise, return to Step 3.

Step 8: Output the optimal value information.

VWCKMA flow is illustrated in Figure 2.

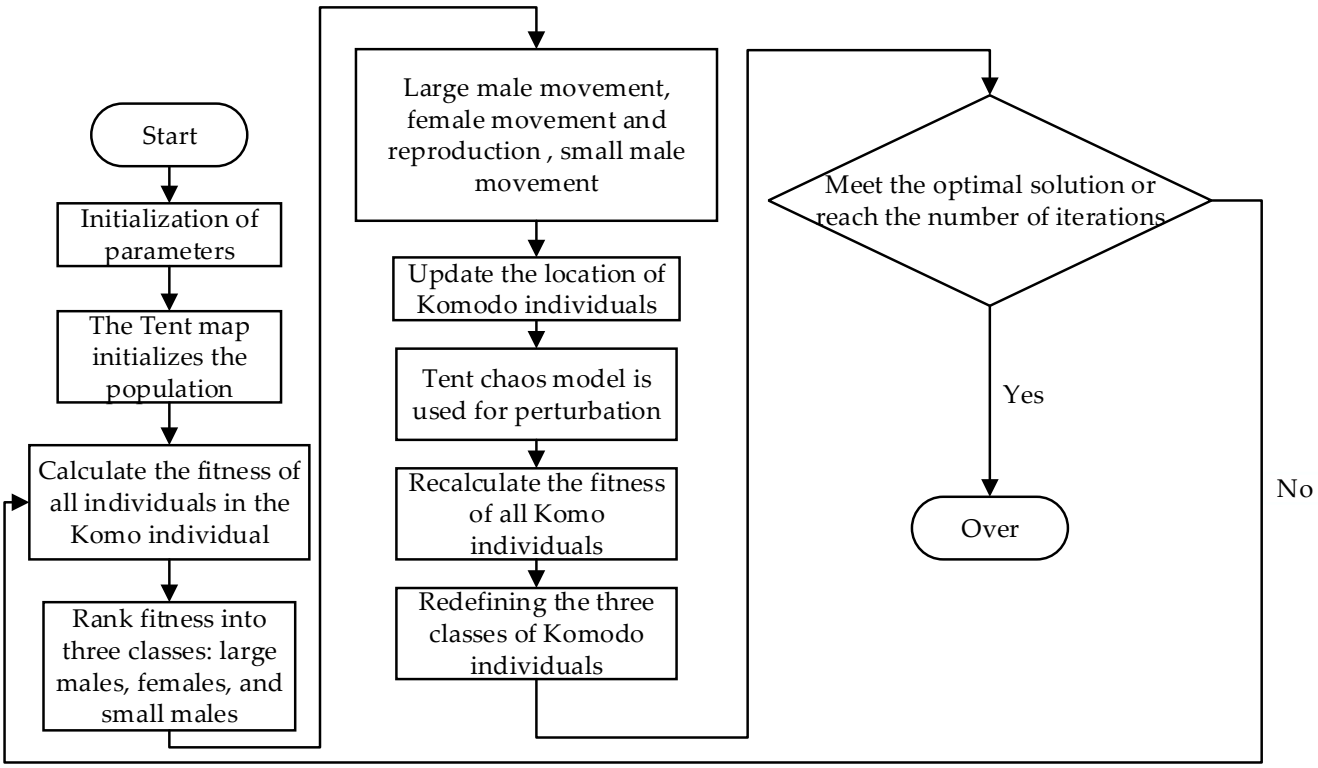


Figure 2. Implementation framework of VWCKMA.

4. Time Complexity Analysis

The time complexity of the standard Komodo Mlipir Algorithm is represented by $O(n \times m \times Max_iteration)$, where n, m and $Max_iteration$ represent the number of individuals in the population, the dimension, and the maximum number of iterations. The following is an analysis of the time complexity of VWCKMA:

(1) The time complexity of Tent mapping initialization population is represented by $O(n \times m)$, then the time complexity of the algorithm introducing Tent mapping initialization population is represented by $O(n \times m \times Max_iteration + n \times m) = O(n \times m \times Max_iteration)$.

(2) Assuming that the time required to update the time-varying inertia weight is represented by t_1, the time complexity of the algorithm after introducing the time-varying inertia weight is represented by $O(n \times m \times Max_iteration + t_1) = O(n \times m \times Max_iteration)$.

(3) The disturbance time complexity of the Tent chaotic model is represented by $O(n \times m)$, so the time complexity of the disturbance algorithm using the Tent chaotic model is represented by $O(n \times m \times Max_iteration + n \times m) = O(n \times m \times Max_iteration)$.

To summarize, As compared with the standard KMA, the proposed VWCKMA has a similar time complexity, which does not increase the algorithm's complexity.

5. Empirical Studies

The purpose of the experiments is to verify the advantages of VWCKMA by comparing it with the standard KMA and other metaheuristic algorithms. Optimization algorithms are used to optimize benchmark functions that are modeled by humans describing real problems they actually encounter.

5.1. Benchmark Functions

A benchmark function is a function that is produced by natural research. It is difficult to solve them using analytical expressions because they are usually diverse and unbiased. Historically, benchmark functions have been an important method for verifying the efficiency, effectiveness, and reliability of algorithms. Test functions are important to verify and compare the performance of optimization algorithms [27].

The benchmark function is a set of functions to test the performance of the evolutionary computing algorithm. The benchmark functions are described in detail by Yao Xin et al. [28]. For the purpose of testing the effectiveness, superiority, and the convergence accuracy of complex benchmark functions in high dimensions of VWCKMA, we choose 20 benchmark functions from the literature [29], including seven unimodal functions ($f_1(x) \sim f_7(x)$), six multimodal functions ($f_8(x) \sim f_{13}(x)$), and seven fixed-dimensional functions ($f_{14}(x) \sim f_{20}(x)$). In addition to the global minimum, the F1 function also has d-dimension local minimum values, which are continuous, concave, and unimodal. The F9 function, called the Rastrigin function, has numerous local minima. In two-dimensional form, the F10 function is characterized by an extremely flat outer area and a large hole in the center. The introduction of the F10 function puts the optimization algorithm at risk of getting stuck in numerous local minima. The F11 function, called the Griewank function, has numerous local minima that are regularly distributed. The rescaled form of the F18 function has a mean of zero and a variance of one and also adds a tiny Gaussian error term to the output. The three kinds of function information are shown in Tables 1–3. The typical 2D versions of the benchmark functions considered in this paper are depicted in Figure 3.

5.2. Experimental Data and Analysis

The KMA, the GWO algorithm, and the PSO algorithm are compared with the VWCKMA in this paper, and numerical simulation experiments are conducted. The involved code is written with MATLAB R2020a software, and the simulation environment is a 64-bit Windows system with an Intel Core i7-12700f 2.10GHz CPU and 32GB of RAM.

In the VWCKMA, KMA, GWO algorithms, and PSO algorithm, the maximum number of iterations is set to 200, and the maximum population size is 200. As a result, benchmark functions of 30-dimensional, 100-dimensional, 500-dimensional, and 1000-dimensional are run independently 30 times and an optimization computation is executed. The optimization solution of the fixed-dimensional functions is depicted in Table 4. As shown in Tables 5–8, the mean and standard deviation of 7 unimodal functions and 6 multimodal functions ($f_1(x) \sim f_{13}(x)$) are calculated by different algorithms under 30 dimensions, 100 dimensions, 500 dimensions, and 1000 dimensions, respectively; the best algorithm is bold and underlined. Figure 4 depicts the convergence curves of some classic test benchmark functions in 30 dimensions to more intuitively reflect the superior performance of the VWCKMA. In order to more intuitively observe the superior performance of VWCKMA

in high-dimensional situations, Figure 5 compares the convergence curves of benchmark functions of VWCKMA and other algorithms in 1000 dimensions.

Table 1. Unimodal test function.

Function	Range	Fmin
$f_1(x) = \sum_{i=1}^{n} x_i^2$	$[-100, 100]$	0
$f_2(x) = \sum_{i=1}^{d} \lvert x_i \rvert + \prod_{i=1}^{d} \lvert x_i \rvert$	$[-10, 100]$	0
$f_3(x) = \sum_{i=1}^{n} \left(\sum_{j-1}^{i} x_j\right)^2$	$[-100, 100]$	0
$f_4(x) = \max_i[\lvert x_i \rvert,\ 1 \le i \le n]$	$[-100, 100]$	0
$f_5(x) = \sum_{i=1}^{n-1} \left\{100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2\right\}$	$[-30, 30]$	0
$f_6(x) = \sum_{i=1}^{n} (x_i + 0.5)^2$	$[-100, 100]$	0
$f_7(x) = \sum_{i=1}^{n} i x_i^4 + random(0, 1)$	$[-1.28, 1.28]$	0

Table 2. Multimodal test function.

Function	Range	Fmin
$f_8(x) = \sum_{i=1}^{n} -x_i \sin(\sqrt{\lvert x_i \rvert})$	$[-500, 500]$	$-418.9829 * Dim$
$f_9(x) = \sum_{i=1}^{n} [x_i^2 - 10\cos(2\pi x_i) + 10]$	$[-5.12, 5.12]$	0
$f_{10}(x) = -20\exp(-0.2\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}) - \exp(\frac{1}{n}\sum_{i=1}^{n}\cos(2\pi x_i)) + 20 + e$	$[-32, 32]$	0
$f_{11}(x) = \frac{1}{4000}\sum_{i=1}^{n} x_i^2 - \prod_{i=1}^{n}\cos(\frac{x_i}{\sqrt{i}}) + 1$	$[-600, 600]$	0
$f_{12}(x) = \frac{\pi}{n}\{10\sin(\pi y_1) + \sum_{i=1}^{n-1}(y_i - 1)^2[1 + 10\sin^2(\pi y_{i+1})] + (y_n - 1)^2\} + \sum_{i=1}^{n} u(x_i, 10, 100, 4)$, $y_i = 1 + \frac{x_i + 1}{4}$, $u(x_i, k, a, m) = \begin{cases} k(y_i - a)^m & ;\ y_i > a \\ 0 & ;\ -a > y_i > a \\ k(-y_i - a)^m & ;\ y_i < -a \end{cases}$	$[-50, 50]$	0
$f_{13}(x) = 0.1\{\sin^2(3\pi y_1) + \sum_{i=1}^{n}(y_i - 1)^2[1 + \sin^2(3\pi y_i + 1)] + (y_n - 1)^2[1 + \sin^2(2\pi y_n)]\} + \sum_{i=1}^{n} u(y_i, 5, 100, 4)$	$[-50, 50]$	0

Table 3. Fixed-dimensional test functions.

Function	Range	Fmin
$f_{14}(x) = (\frac{1}{500} + \sum_{j=1}^{25}\frac{1}{j + \sum_{i=1}^{2}(y_i - a_{ij})^6})^{-1}$	$[-65, 65]$	1
$f_{15}(x) = \sum_{i=1}^{11}\left(a_i - \frac{x_1(b_i^2 + b_i x_2)}{b_i^2 + b_i x_3 + x_4}\right)^2$	$[-5, 5]$	0.00030
$f_{16}(x) = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 - 4x_2^2 + 4x_2^4$	$[-5, 5]$	-1.0316
$f_{17}(x) = \left\{x_2 - \frac{5.1}{4\pi^2}x_1^2 + \frac{5}{\pi}x_1 - 6\right\}^2 + 10(1 - \frac{1}{8\pi})\cos x_1 + 10$	$[-5, 5]$	0.398
$f_{18}(x) = [1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3y_2^2) \times [30 + (2x_1 - 3x_2)^2 \times (18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2)]]$	$[-2, 2]$	3
$f_{19}(x) = -\sum_{i=4}^{4} c_i \cdot \exp(-\sum_{j=1}^{3} a_{ij}(y_j - p_{ij})^2)$	$[1, 3]$	-3.86
$f_{20}(y) = -\sum_{i=4}^{4} c_i \cdot \exp(-\sum_{j=1}^{6} a_{ij}(y_j - p_{ij})^2)$	$[0, 1]$	-3.32

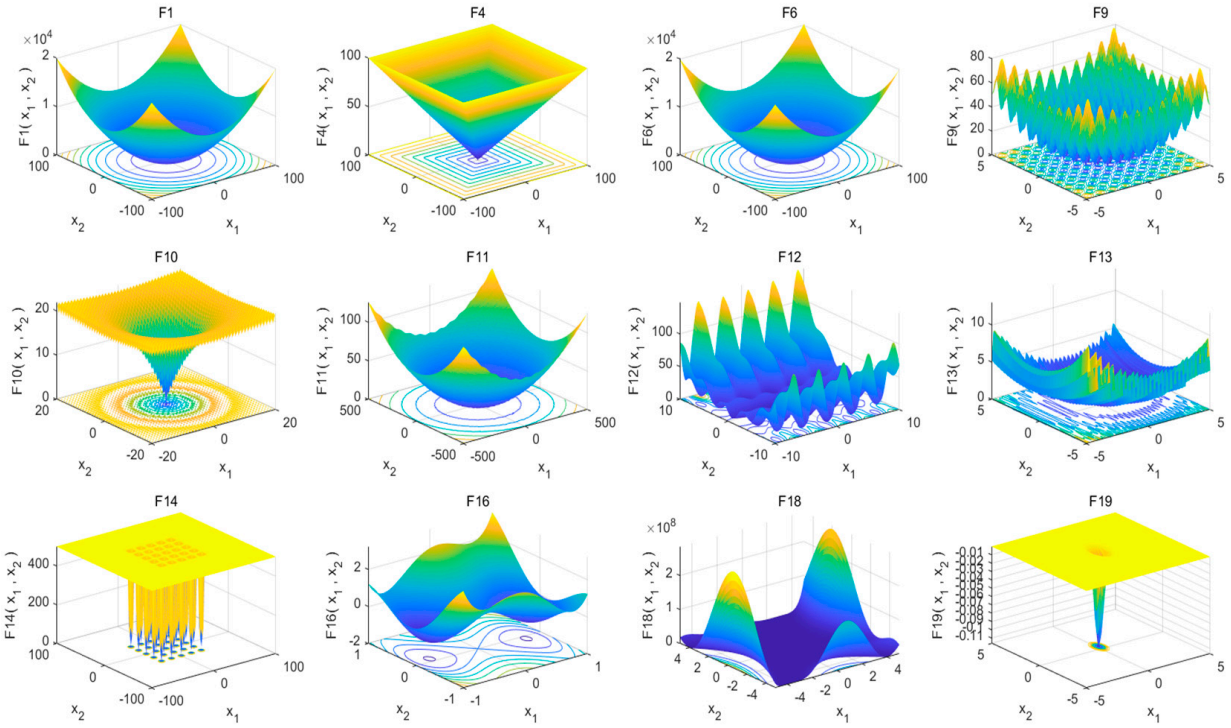


Figure 3. 2D representations of a typical benchmark mathematical functions.

Table 4. Comparison of different algorithms in solving the fixed-dimensional test functions.

	PSO		GWO		KMA		VWCKMA	
	Mean	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**
$f_{14}(x)$	$\mathbf{9.98 \times 10^{-1}}$	$\mathbf{0.00 \times 10^{0}}$	1.23×10^{0}	6.21×10^{-1}	6.20×10^{0}	3.59×10^{0}	$\mathbf{9.98 \times 10^{-1}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{15}(x)$	1.29×10^{-3}	3.62×10^{-3}	1.68×10^{-3}	5.08×10^{-3}	5.51×10^{-2}	5.10×10^{-2}	$\mathbf{3.07 \times 10^{-4}}$	$\mathbf{3.80 \times 10^{-9}}$
$f_{16}(x)$	$\mathbf{-1.0 \times 10^{0}}$	6.52×10^{-16}	$\mathbf{-1.0 \times 10^{0}}$	1.48×10^{-8}	-9.4×10^{-1}	1.27×10^{-1}	$\mathbf{-1.0 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{17}(x)$	$\mathbf{3.98 \times 10^{-1}}$	$\mathbf{0.0 \times 10^{0}}$	$\mathbf{3.98 \times 10^{-1}}$	4.19×10^{-4}	4.94×10^{-1}	3.04×10^{-1}	$\mathbf{3.98 \times 10^{-1}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{18}(x)$	$\mathbf{3.00 \times 10^{0}}$	6.99×10^{-16}	$\mathbf{3.00 \times 10^{0}}$	3.61×10^{-6}	1.46×10^{1}	1.77×10^{1}	$\mathbf{3.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{19}(x)$	$\mathbf{-3.8 \times 10^{0}}$	2.71×10^{-15}	$\mathbf{-3.8 \times 10^{0}}$	9.66×10^{-4}	-3.7×10^{0}	3.01×10^{-1}	$\mathbf{-3.8 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{20}(x)$	-3.2×10^{0}	7.57×10^{-2}	-3.2×10^{0}	6.64×10^{-2}	-2.3×10^{0}	7.57×10^{-1}	$\mathbf{-3.3 \times 10^{0}}$	$\mathbf{2.77 \times 10^{-8}}$

Table 5. Comparison of different algorithms in solving the multimodal and unimodal test functions at 30 dimensions.

	PSO		GWO		KMA		VWCKMA	
	Mean	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**
$f_1(x)$	6.34×10^{-2}	8.92×10^{-2}	4.94×10^{-17}	5.49×10^{-17}	6.7×10^{-135}	2.5×10^{-134}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_2(x)$	6.51×10^{0}	7.14×10^{0}	1.45×10^{-10}	7.22×10^{-11}	3.95×10^{-72}	1.01×10^{-71}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_3(x)$	4.04×10^{3}	4.54×10^{3}	2.71×10^{-4}	2.32×10^{-4}	1.2×10^{-107}	6.5×10^{-107}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_4(x)$	4.22×10^{0}	1.31×10^{0}	1.73×10^{-4}	8.46×10^{-5}	1.02×10^{-65}	3.33×10^{-65}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_5(x)$	6.90×10^{2}	1.32×10^{3}	$\mathbf{2.66 \times 10^{1}}$	$\mathbf{0.86 \times 10^{0}}$	2.89×10^{1}	1.13×10^{2}	2.84×10^{1}	0.14×10^{0}
$f_6(x)$	1.88×10^{-1}	6.68×10^{-1}	1.13×10^{-1}	1.68×10^{-1}	5.66×10^{0}	7.31×10^{-1}	$\mathbf{9.91 \times 10^{-2}}$	$\mathbf{5.90 \times 10^{-2}}$
$f_7(x)$	2.31×10^{-2}	8.39×10^{-3}	$\mathbf{9.46 \times 10^{-4}}$	$\mathbf{4.29 \times 10^{-4}}$	5.00×10^{-3}	1.12×10^{-2}	2.59×10^{-3}	3.06×10^{-3}
$f_8(x)$	-8.9×10^{3}	7.94×10^{2}	-6.6×10^{3}	7.96×10^{2}	-2.6×10^{3}	8.71×10^{2}	$\mathbf{-1.2 \times 10^{4}}$	$\mathbf{6.41 \times 10^{2}}$
$f_9(x)$	4.91×10^{1}	1.53×10^{1}	5.30×10^{0}	2.531×10^{0}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{10}(x)$	1.11×10^{0}	6.73×10^{-1}	1.13×10^{-9}	5.52×10^{-10}	2.01×10^{0}	6.12×10^{0}	$\mathbf{8.88 \times 10^{-16}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{11}(x)$	7.00×10^{-2}	6.93×10^{-2}	2.24×10^{-3}	5.19×10^{-3}	0.00×10^{0}	0.00×10^{0}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{12}(x)$	7.76×10^{-1}	7.03×10^{-1}	1.07×10^{-2}	6.49×10^{-3}	1.22×10^{0}	4.03×10^{-1}	$\mathbf{5.68 \times 10^{-3}}$	$\mathbf{8.82 \times 10^{-3}}$
$f_{13}(x)$	2.10×10^{-1}	5.44×10^{-1}	1.23×10^{-1}	1.15×10^{-1}	2.94×10^{0}	1.36×10^{-1}	$\mathbf{4.34 \times 10^{-2}}$	$\mathbf{3.12 \times 10^{-2}}$

Table 6. Comparison of different algorithms in solving the multimodal and unimodal test functions at 100 dimensions.

	PSO		GWO		KMA		VWCKMA	
	Mean	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**
$f_1(x)$	6.86×10^{3}	4.31×10^{3}	3.99×10^{-6}	1.31×10^{-6}	6.8×10^{-137}	2.2×10^{-136}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_2(x)$	2.50×10^{2}	4.62×10^{1}	3.32×10^{-4}	5.40×10^{-5}	1.33×10^{-72}	1.74×10^{-72}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_3(x)$	1.02×10^{5}	2.24×10^{5}	8.12×10^{2}	4.24×10^{2}	2.2×10^{-104}	7.0×10^{-104}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_4(x)$	4.62×10^{1}	3.76×10^{0}	9.43×10^{-1}	5.68×10^{-1}	4.36×10^{-68}	8.38×10^{-68}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_5(x)$	8.96×10^{5}	2.36×10^{5}	9.82×10^{1}	3.64×10^{-1}	9.89×10^{1}	7.30×10^{-2}	$\mathbf{9.80 \times 10^{1}}$	$\mathbf{6.84 \times 10^{-2}}$
$f_6(x)$	1.36×10^{4}	7.92×10^{3}	7.24×10^{0}	8.24×10^{-1}	1.98×10^{1}	1.04×10^{0}	$\mathbf{1.84 \times 10^{0}}$	$\mathbf{7.23 \times 10^{-1}}$
$f_7(x)$	1.99×10^{1}	1.85×10^{1}	5.40×10^{-3}	1.88×10^{-3}	$\mathbf{2.13 \times 10^{-3}}$	$\mathbf{2.62 \times 10^{-3}}$	2.14×10^{-3}	1.27×10^{-3}
$f_8(x)$	-2.2×10^{4}	1.69×10^{3}	-1.8×10^{4}	9.86×10^{2}	-7.6×10^{3}	2.38×10^{3}	$\mathbf{-3.6 \times 10^{4}}$	$\mathbf{1.54 \times 10^{3}}$
$f_9(x)$	5.02×10^{2}	5.04×10^{1}	2.85×10^{1}	1.22×10^{1}	4.58×10^{1}	1.45×10^{2}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{10}(x)$	1.10×10^{1}	2.99×10^{0}	2.12×10^{-4}	5.23×10^{-5}	1.99×10^{0}	6.31×10^{0}	$\mathbf{8.88 \times 10^{-16}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{11}(x)$	8.46×10^{1}	5.66×10^{1}	2.90×10^{-3}	9.15×10^{-3}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{12}(x)$	4.44×10^{3}	9.29×10^{3}	1.59×10^{-1}	3.49×10^{-2}	1.53×10^{0}	1.57×10^{0}	$\mathbf{1.39 \times 10^{-2}}$	$\mathbf{4.41 \times 10^{-3}}$
$f_{13}(x)$	5.10×10^{5}	7.95×10^{5}	5.58×10^{0}	7.08×10^{-1}	9.45×10^{0}	1.66×10^{0}	$\mathbf{4.63 \times 10^{-1}}$	$\mathbf{1.38 \times 10^{-1}}$

Table 7. Comparison of different algorithms in solving the multimodal and unimodal test functions at 500 dimensions.

	PSO		GWO		KMA		VWCKMA	
	Mean	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**
$f_1(x)$	4.57×10^{5}	4.58×10^{3}	1.37×10^{1}	9.45×10^{-1}	3.5×10^{-135}	7.6×10^{-135}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_2(x)$	1.89×10^{3}	2.99×10^{1}	2.68×10^{0}	1.63×10^{-1}	8.21×10^{-70}	1.05×10^{-69}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_3(x)$	8.04×10^{4}	1.51×10^{6}	1.24×10^{3}	1.38×10^{3}	2.1×10^{-119}	4.6×10^{-119}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_4(x)$	6.81×10^{1}	2.51×10^{0}	5.83×10^{1}	6.30×10^{5}	1.15×10^{-65}	2.20×10^{-65}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_5(x)$	6.22×10^{5}	3.48×10^{5}	9.80×10^{4}	7.24×10^{-1}	9.89×10^{1}	6.63×10^{-2}	$\mathbf{9.80 \times 10^{1}}$	$\mathbf{4.60 \times 10^{-2}}$
$f_6(x)$	2.51×10^{3}	1.38×10^{4}	7.60×10^{0}	1.17×10^{0}	1.91×10^{1}	1.50×10^{0}	$\mathbf{1.28 \times 10^{0}}$	$\mathbf{2.00 \times 10^{-1}}$
$f_7(x)$	8.20×10^{3}	1.07×10^{3}	1.33×10^{-1}	7.97×10^{-3}	$\mathbf{2.54 \times 10^{-3}}$	2.39×10^{-3}	$\mathbf{2.54 \times 10^{-3}}$	$\mathbf{2.21 \times 10^{-3}}$
$f_8(x)$	-5.9×10^{4}	2.97×10^{3}	-6.4×10^{4}	2.71×10^{3}	-1.2×10^{5}	6.03×10^{3}	$\mathbf{-1.3 \times 10^{5}}$	$\mathbf{9.01 \times 10^{3}}$
$f_9(x)$	4.96×10^{3}	1.03×10^{-2}	5.35×10^{2}	7.50×10^{1}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{10}(x)$	1.94×10^{1}	2.54×10^{-1}	3.70×10^{-1}	8.72×10^{-2}	2.01×10^{0}	6.12×10^{0}	$\mathbf{8.88 \times 10^{-16}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{11}(x)$	4.18×10^{3}	1.60×10^{2}	9.55×10^{-1}	1.18×10^{-1}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{12}(x)$	5.77×10^{2}	1.30×10^{3}	1.33×10^{-1}	2.87×10^{-2}	1.22×10^{0}	1.45×10^{-1}	$\mathbf{1.86 \times 10^{-2}}$	$\mathbf{7.02 \times 10^{-3}}$
$f_{13}(x)$	5.12×10^{5}	1.21×10^{6}	5.54×10^{0}	7.53×10^{-1}	9.94×10^{0}	1.30×10^{-1}	$\mathbf{4.19 \times 10^{-1}}$	$\mathbf{5.16 \times 10^{-2}}$

Table 8. Comparison of different algorithms in solving the multimodal and unimodal test functions at 1000 dimensions.

	PSO		GWO		KMA		VWCKMA	
	Mean	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**	**Mean**	**Std.**
$f_1(x)$	1.31×10^{6}	6.15×10^{4}	4.86×10^{2}	6.86×10^{1}	3.5×10^{-133}	6.5×10^{-133}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_2(x)$	2.23×10^{3}	2.33×10^{1}	3.57×10^{1}	3.34×10^{0}	1.27×10^{-48}	2.20×10^{-48}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_3(x)$	1.03×10^{7}	1.14×10^{6}	1.68×10^{6}	3.75×10^{5}	3.7×10^{-102}	8.2×10^{-102}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_4(x)$	9.91×10^{1}	6.41×10^{-1}	7.45×10^{1}	2.14×10^{0}	4.36×10^{-65}	5.67×10^{-65}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_5(x)$	3.69×10^{9}	1.36×10^{8}	4.13×10^{4}	1.02×10^{4}	9.98×10^{2}	2.89×10^{-1}	$\mathbf{9.91 \times 10^{2}}$	$\mathbf{4.25 \times 10^{-1}}$
$f_6(x)$	1.32×10^{6}	3.36×10^{4}	7.35×10^{2}	9.72×10^{1}	2.17×10^{2}	1.02×10^{1}	$\mathbf{3.44 \times 10^{1}}$	$\mathbf{6.53 \times 10^{0}}$
$f_7(x)$	5.46×10^{4}	5.30×10^{3}	1.45×10^{0}	5.25×10^{-1}	$\mathbf{8.23 \times 10^{-4}}$	$\mathbf{8.94 \times 10^{-4}}$	2.68×10^{-3}	3.37×10^{-3}
$f_8(x)$	-8.9×10^{4}	3.93×10^{3}	-1.1×10^{5}	4.72×10^{3}	$\mathbf{-2.8 \times 10^{5}}$	$\mathbf{1.55 \times 10^{4}}$	-2.4×10^{5}	1.21×10^{4}
$f_9(x)$	1.16×10^{4}	2.42×10^{2}	1.63×10^{3}	1.74×10^{2}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{10}(x)$	1.98×10^{1}	8.90×10^{-2}	2.52×10^{0}	8.36×10^{-2}	1.22×10^{1}	1.11×10^{1}	$\mathbf{8.88 \times 10^{-16}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{11}(x)$	1.15×10^{4}	4.75×10^{-2}	5.72×10^{0}	5.99×10^{-1}	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$	$\mathbf{0.00 \times 10^{0}}$
$f_{12}(x)$	7.29×10^{9}	6.97×10^{8}	7.58×10^{0}	1.40×10^{0}	1.01×10^{0}	2.23×10^{-1}	$\mathbf{3.70 \times 10^{-3}}$	$\mathbf{1.21 \times 10^{-2}}$
$f_{13}(x)$	1.60×10^{10}	1.54×10^{9}	5.59×10^{2}	4.10×10^{1}	100×10^{0}	6.53×10^{-3}	$\mathbf{9.07 \times 10^{0}}$	$\mathbf{1.26 \times 10^{0}}$

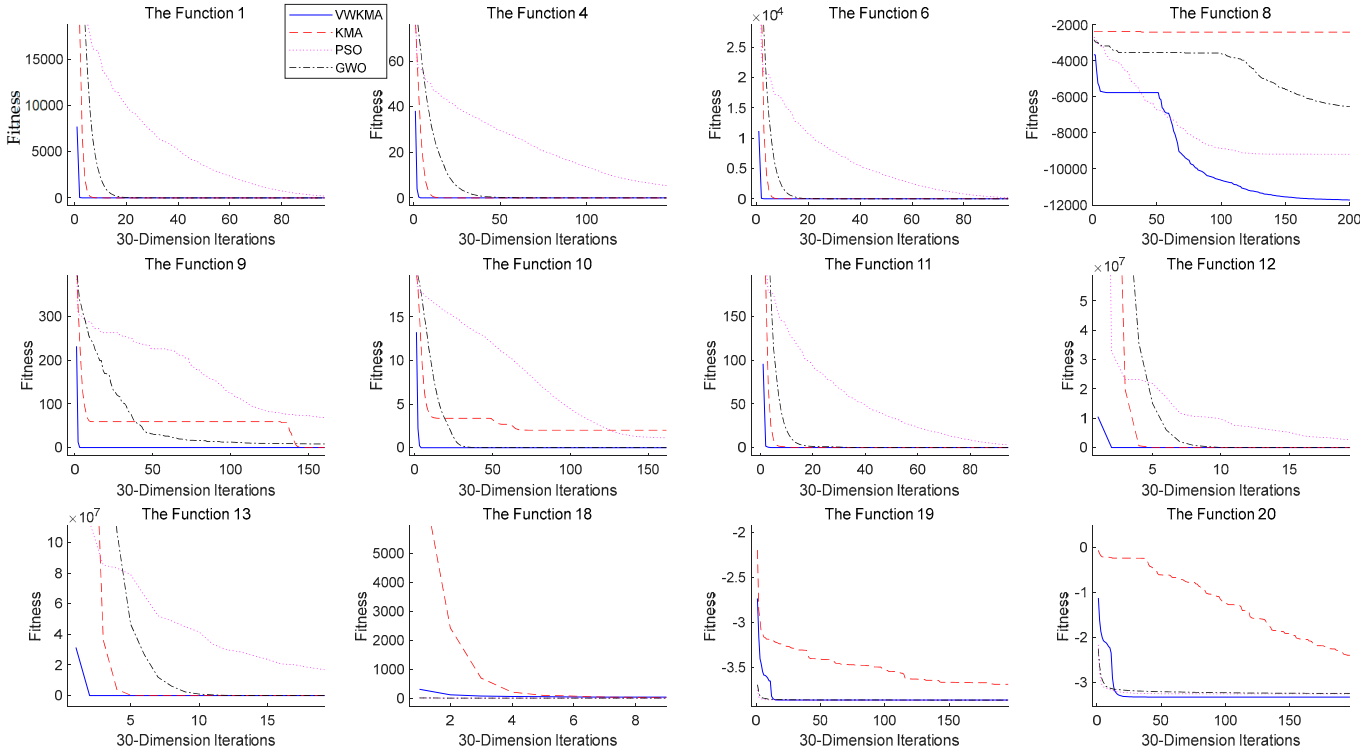


Figure 4. Comparison of the convergence curves obtained by VWCKMA and other algorithms for some benchmark functions in 30 dimensions.

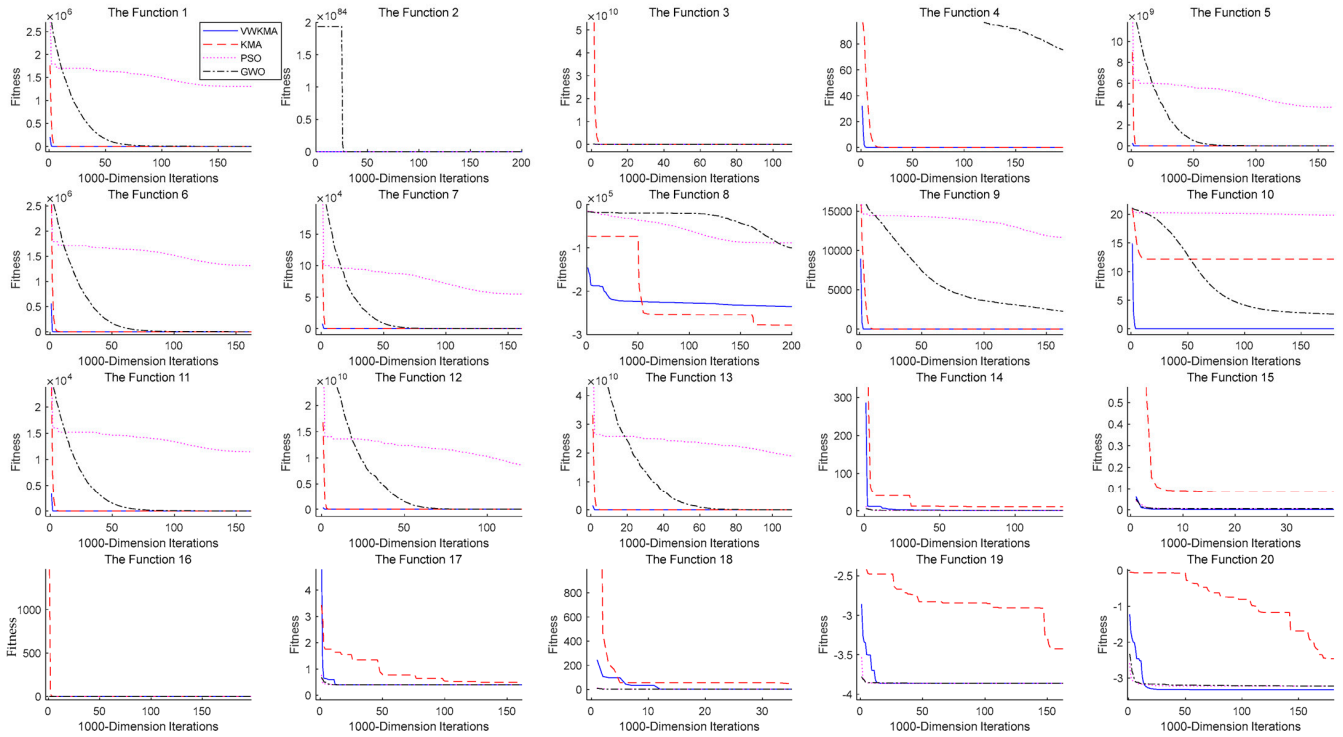


Figure 5. Comparison of the convergence curves obtained by different algorithms for benchmark functions in 1000 dimensions.

The fixed-dimensional function can be a good test of the exploratory ability of the algorithm. According to the results in Table 4, VWCKMA performs well in terms of con-

vergence accuracy, with the smallest standard deviation in the fixed-dimensional function ($f_{14}(x) \sim f_{20}(x)$), implying that the VWCKMA has superior convergence accuracy.

Functions $f_1(x) \sim f_7(x)$ are unimodal which have only one global optimal solution and are used to evaluate the exploitation ability of the algorithm. Unlike unimodal functions, multimodal functions ($f_8(x) \sim f_{13}(x)$) contain many local optima, the number of which varies with dimension. Therefore, this multimodal test benchmark function can effectively evaluate the exploration ability of the optimization algorithm. As can be seen from Tables 5–8, the VWCKMA has strong advantages compared with other metaheuristic algorithms. As shown in Table 5, the VWCKMA is the most efficient optimizer for $f_1(x) \sim f_4(x)$, $f_6(x)$, and $f_8(x) \sim f_{13}(x)$, and is at least the second-best optimizer among the $f_5(x)$ and $f_7(x)$ test functions at the 30 dimension. Consequently, the VWCKMA has terrific exploitation ability. In Table 6, only function 7 is slightly inferior to the KMA at 100 dimensions. The optimal convergence value of the VWCKMA for $f_7(x)$ is 1×10^{-5} less than that of the KMA. The difference between the two is very small and can be ignored. It can be seen from Table 7 that the VWCKMA can converge to the theoretical optimal value of the function for $f_1(x) \sim f_4(x)$, $f_9(x)$, and $f_{10}(x)$. The convergence accuracy of the remaining functions is the highest compared to other algorithms. For the simulation experiments in 1000 dimensions shown in Table 8, in 1000 dimensions, the VWCKMA still has a high-strength convergence ability. Among the $f_1(x) \sim f_6(x)$ and $f_8(x) \sim f_{20}(x)$, the VWCKMA is the best optimization algorithm. Compared with the $f_7(x)$, there is little difference with the KMA. Other metaheuristic algorithms fail to converge in 1000 dimensions.

As can be seen from Figure 4, compared with other algorithms, VWCKMA has fewer convergence iterations and better convergence accuracy. The convergence speed and accuracy of the VWCKMA are better than the standard KMA. In the $f_8(x) \sim f_{10}(x)$ and $f_{14}(x) \sim f_{20}(x)$, when KMA falls into local optimal value, VWCKMA can effectively jump out of local optimal value. The results indicate that VWCKMA has high convergence accuracy and fast convergence speed.

To conclude, VWCKMA is clearly superior to the four swarm intelligence algorithms in terms of convergence accuracy and convergence speed, as well as in terms of optimization calculation of unimodal functions and multimodal functions. In addition to its strong ability to perform global exploration and local search as well as fast convergence speeds. For high-dimensional or even thousand-dimensional conditions, it still is also highly resistant to local optimal solutions. As a result, it can jump out of the local optimal solution and find the optimal value of the function.

6. Practical Application

Currently, pollutant prediction models mainly consist of the following three categories: physical models [30], statistical models [31], and artificial intelligence models [32]. In this paper, the VWCKMA is applied to the prediction of PM2.5 in Chengdu. We selected five models of traditional BP neural networks [33], particle swarm optimization neural networks [34] (PSO-BP), KMA optimization BP neural networks (KMA-BP), VWCKMA optimization BP neural networks (VWCKMA-BP), and Random Forest Algorithm for Traditional Machine Learning [35] to predict air pollution PM2.5 in Chengdu using MATLAB simulation experiments. A fair comparison of the five prediction models is made. The structure of the BP neural network is 5-10-5-1, the transfer function from the input layer to the hidden layer is Tansig function, the transfer function from the hidden layer to the output layer is Poslin function, and the learning rate is 0.4. In the random forest algorithm, the number of leaf nodes is 5, and the number of trees is 200. The sample data were collected from the China air quality online monitoring and analysis platform (https://www.aqistudy.cn, accessed on 12 October 2022). In Chengdu, 912 pieces of data were selected for analysis from 1 January 2020 to 30 June 2022. There is detailed information regarding date, AQI, quality grade, PM2.5, PM10, CO, NO_2, SO_2, and O_3. For a summary of the data, please refer to Table 9. The following data are listed due to a lack of space.

Table 9. Original data samples.

Date	AQI	Quality	PM2.5	PM10	CO	NO_2	SO_2	O_3
1 January 2020	135	Light	103	124	45	0.9	6	27
2 January 2020	135	Light	103	132	57	1	10	22
3 January 2020	105	Light	79	102	57	1	6	56
4 January 2020	118	Light	89	119	67	1.1	8	16
...	...	...	...	...	...	...	...	...
29 June 2022	106	Light	20	39	24	5	0.5	166
30 June 2022	82	good	26	50	35	5	0.7	138

As shown in Table 9, the unit of the 6 pollutant concentration data is $\mu g/m^3$, and the air quality AQI is a dimensionless index. In an all-around comparison, prediction value is compared with real value. The evaluation indicators are introduced, which include the mean relative error (*MAPE*), the mean square error (*MSE*), the mean absolute error (*MAE*), the root mean square error (*RMSE*), and prediction accuracy (*ACC*). As the fitness function of the algorithm, the mean square error function is used in all prediction models. Each of the four indicators is defined mathematically as follows:

$$MAPE = \frac{1}{N}\sum_{j=1}^{N}\frac{\left|y_i - y_j^*\right|}{y_i} \tag{16}$$

$$MSE = \frac{1}{N}\sum_{j=1}^{N}(y_i - y_j^*)^2 \tag{17}$$

$$MAE = \frac{1}{N}\sum_{j=1}^{N}\left|y_i - y_j^*\right| \tag{18}$$

$$RMSE = \sqrt{\frac{1}{N}\sum_{j=1}^{N}(y_i - y_j^*)^2} \tag{19}$$

where N represents the number of test samples, y_i represents the true value of the air quality index, and y_j^* represents the predicted value of the air quality index.

Results Analysis

A total of 70% of the 912 pieces of data collected from 1 January 2020 to 30 June 2022 are selected as training sets and 30% as test sets. Figure 6 illustrates the prediction comparison results of the five predictive models. The predicted value model of the VWCKMA-BP is compared to the real value, as shown in Figure 7. Table 10 presents the statistical results of *MAPE*, *MSE*, *MAE*, *RMSE*, and prediction accuracy (1-daily relative error rate = daily prediction accuracy rate, and then calculates the arithmetic mean value of all prediction samples) for the five predictive models. As shown in Table 10, different optimization strategies have a significant impact on the prediction of BP neural networks. With the VWCKMA, the accuracy rate of the improved BP neural network algorithm reaches 85.085%.

Table 10. Statistical results of indicators.

Model	*MAE*/($\mu g/m^3$)	*MSE*/($\mu g/m^3$)	*RMSE*/($\mu g/m^3$)	*MAPE*/%	*ACC*/%
BPNN	12.7518	280.2218	16.7398	34.7621	65.238
PSO-BP	9.9420	193.3857	13.9063	26.0763	73.923
KMA-BP	6.4277	64.3231	8.0202	19.7768	80.223
RandomForest	6.0028	66.7176	8.1681	16.2575	83.743
VWCKMA-BP	5.2226	56.1275	7.4918	14.9152	85.085

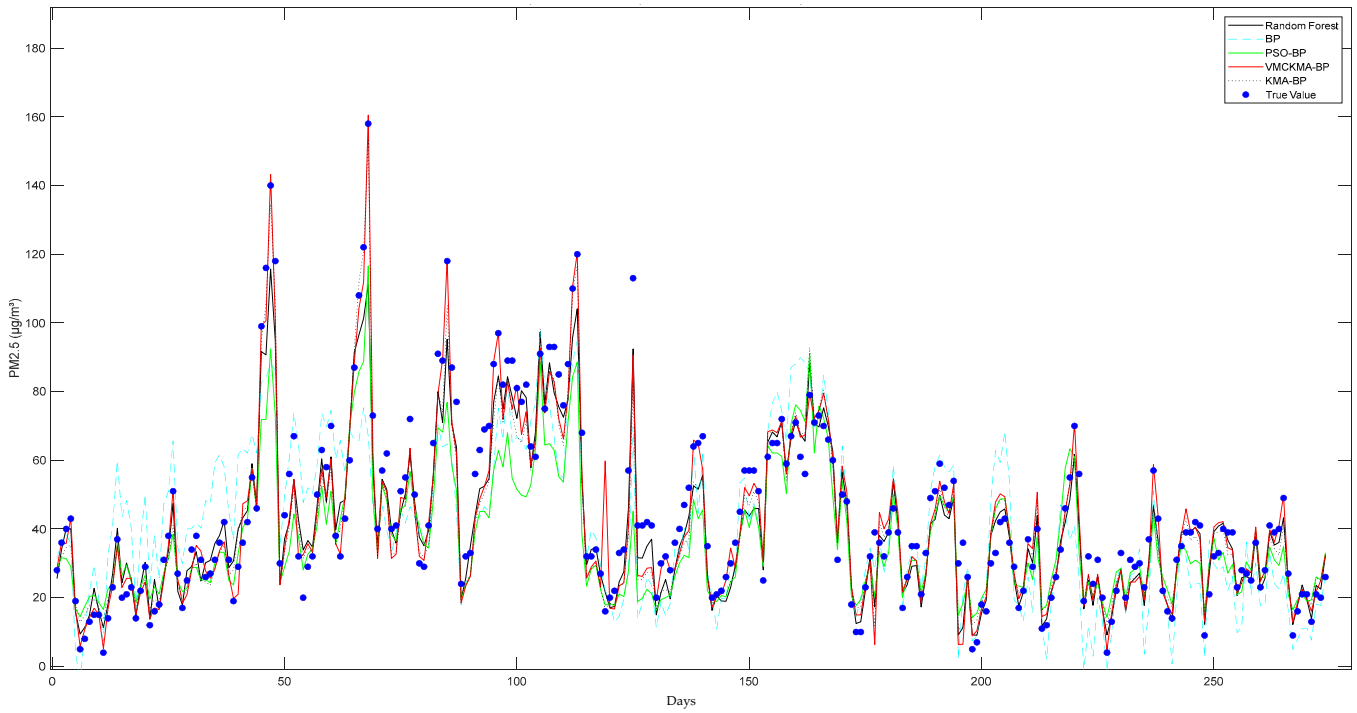


Figure 6. Comparison of real and predicted values of different prediction models.

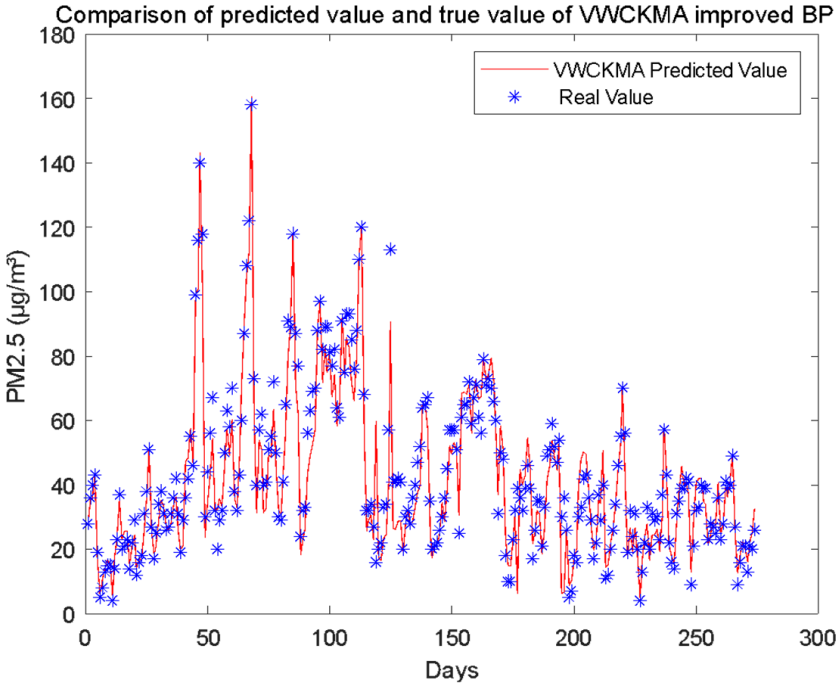


Figure 7. Comparison of predicted and real value of the improved BP neural network by VWCKMA.

As shown in Figure 7, the BP neural network model optimized by the VWKMA is closer to the true value, and the optimization accuracy is significantly improved by the VWCKMA. Among the five models, only the VWCKMA-optimized neural network model (VWCKMA-BP) produces an average absolute error *MAE* that is lower than 6. The root means square error, *MSE*, is less than 60, the mean square error, *MAPE*, is less than 15%, and the accuracy rate reaches 85.085%. Figure 8 shows the comparative convergence curves for KMA, PSO, and VWCKMA. Due to the initialization population of Tent mapping, VWCKMA has better fitness values than the other two models VWCKMA is close to

convergence after approximately 30 iterations, and other algorithms have a higher fitness value than VWCKMA. According to the above results, the improved KMA optimization neural network model is effective and feasible for predicting the PM2.5 index.

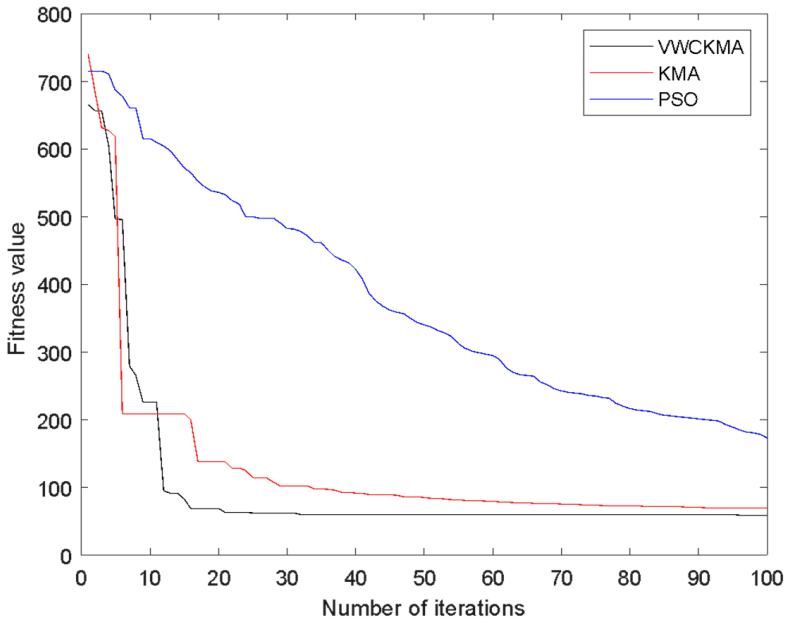


Figure 8. Iterative convergence curve of the algorithm.

Author Contributions: Validation, writing original draft, M.Z.; Investigation, Writing—review and editing; L.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Ministry of Education New Generation Information Technology Innovation Project 2021, grant number 2021ITA05050.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Lin, S.; Kernighan, B.W. An effective heuristic algorithm for the traveling-salesman problem. *Oper. Res.* **1973**, *21*, 498–516. [CrossRef]
2. Grefenstette, J.; Gopal, R.; Rosmaita, B.J.; Gucht, D.V. Genetic algorithms for the traveling salesman problem. In Proceedings of the the First International Conference on Genetic Algorithms and Their Applications, Pittsburgh, PA, USA, 24–26 July 1985; pp. 160–168.
3. Hu, Y.; Yang, S.X. A knowledge based genetic algorithm for path planning of a mobile robot. In Proceedings of the IEEE International Conference on Robotics and Automation, New Orleans, LA, USA, 26 April–1 May 2004; Volume 2004, pp. 4350–4355.
4. Ismail, A.; Sheta, A.; Al-Weshah, M. A mobile robot path planning using genetic algorithm in static environment. *J. Comput. Sci.* **2008**, *4*, 341–344.
5. Yin, L.; Li, X.; Gao, L.; Lu, C.; Zhang, Z. A novel mathematical model and multi-objective method for the low-carbon flexible job shop scheduling problem. *Sustain. Comput. Inform. Syst.* **2017**, *13*, 15–30. [CrossRef]
6. Jiang, T.; Deng, G. Optimizing the low-carbon flexible job shop scheduling problem considering energy consumption. *IEEE Access* **2018**, *6*, 46346–46355. [CrossRef]
7. Poli, R.; Kennedy, J.; Blackwell, T. Particle swarm optimization. *Swarm Intell.* **2007**, *1*, 33–57. Available online: http://d.wanfangdata.com.cn/Periodical_dqkx201507013.aspx (accessed on 10 September 2022). [CrossRef]
8. Shi, Y.; Eberhart, R. A modified particle swarm optimizer. In Proceedings of the IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No. 98TH8360), Piscataway, NJ, USA, 4–9 May 1998; pp. 69–73.
9. Dorigo, M.; Birattari, M.; Stutzle, T. Ant colony optimization. *IEEE Comput. Intell. Mag.* **2006**, *1*, 28–39. [CrossRef]
10. Houck, C.R.; Joines, J.; Kay, M.G. A genetic algorithm for function optimization: A Matlab implementation. *Ncsu-Ie Tr* **1995**, *95*, 1–10.
11. Mirjalili, S. *Genetic Algorithm //Evolutionary Algorithms and Neural Networks: Theory and Applications*; Springer International Publishing: Cham, Switzerland, 2019; pp. 43–55.

12. Xue, J.; Shen, B. A novel swarm intelligence optimization approach: Sparrow search algorithm. *Syst. Sci. Control Eng.* **2020**, *8*, 22–34. [CrossRef]
13. Mirjalili, S.; Mirjalili, S.M.; Lewis, A. Grey wolf optimizer. *Adv. Eng. Softw.* **2014**, *69*, 46–61. [CrossRef]
14. Mirjalili, S.; Lewis, A. The whale optimization algorithm. *Adv. Eng. Softw.* **2016**, *95*, 51–67. [CrossRef]
15. Agushaka, J.O.; Ezugwu, A.E.; Abualigah, L. Dwarf mongoose optimization algorithm. *Comput. Methods Appl. Mech. Eng.* **2022**, *391*, 114570. [CrossRef]
16. Li, S.; Chen, H.; Wang, M.; Heidari, A.A.; Mirjalili, S. Slime mould algorithm: A new method for stochastic optimization. *Future Gener. Comput. Syst.* **2020**, *111*, 300–323. [CrossRef]
17. Sulaiman, M.H.; Mustaffa, Z.; Saari, M.M.; Daniyal, H. Barnacles mating optimizer: A new bio-inspired algorithm for solving engineering optimization problems. *Eng. Appl. Artif. Intell.* **2020**, *87*, 103330. [CrossRef]
18. Muthiah-Nakarajan, V.; Noel, M.M. Galactic swarm optimization: A new global optimization metaheuristic inspired by galactic motion. *Appl. Soft Comput.* **2016**, *38*, 771–787. [CrossRef]
19. Waharte, S.; Trigoni, N. Supporting search and rescue operations with UAVs. In Proceedings of the International Conference on Emerging Security Technologies, Canterbury, UK, 6–7 September 2010; pp. 142–147.
20. Suyanto, S.; Ariyanto, A.A.; Ariyanto, A.F. Komodo Mlipir Algorithm. *Appl. Soft Comput.* **2022**, *114*, 108043. [CrossRef]
21. Zhang, Z.; Ding, S.; Sun, Y. A support vector regression model hybridized with chaotic krill herd algorithm and empirical mode decomposition for regression task. *Neurocomputing* **2020**, *410*, 185–201. [CrossRef]
22. Sun, W.; Xu, Y. Using a back propagation neural network based on improved particle swarm optimization to study the influential factors of carbon dioxide emissions in Hebei Province, China. *J. Clean. Prod.* **2016**, *112*, 1282–1291. [CrossRef]
23. Gao, Y.; Xie, L.; Zhang, Z.; Fan, Q. Twin support vector machine based on improved artificial fish swarm algorithm with application to flame recognition. *Appl. Intell.* **2020**, *50*, 2312–2327. [CrossRef]
24. Goluguri, N.; Devi, K.S.; Srinivasan, P. Rice-net: An efficient artificial fish swarm optimization applied deep convolutional neural network model for identifying the Oryza sativa diseases. *Neural Comput. Appl.* **2021**, *33*, 5869–5884. [CrossRef]
25. Wang, B.; Xue, B.; Zhang, M. Particle swarm optimisation for evolving deep neural networks for image classification by evolving and stacking transferable blocks. In Proceedings of the IEEE Congress on Evolutionary Computation (CEC), Glasgow, UK, 19–24 July 2020; pp. 1–8.
26. Shan, L.; Qiang, H.; Li, J.; Wang, Z. Chaotic optimization algorithm based on Tent map. *Control Decis.* **2005**, *20*, 179–182.
27. Jamil, M.; Yang, X.-S. A literature survey of benchmark functions for global optimization problems. *arXiv* **2013**, arXiv:1308.4008 2013.
28. Yao, X.; Liu, Y.; Lin, G. Evolutionary programming made faster. *IEEE Trans. Evol. Comput.* **1999**, *3*, 82–102.
29. Chopra, N.; Ansari, M.M. Golden jackal optimization: A novel nature-inspired optimizer for engineering applications. *Expert Syst. Appl.* **2022**, *198*, 116924. [CrossRef]
30. Huang, K.; Xiao, Q.; Meng, X.; Geng, G.; Wang, Y.; Lyapustin, A.; Gu, D.; Liu, Y. Predicting monthly high-resolution PM2.5 concentrations with random forest model in the North China Plain. *Environ. Pollut.* **2018**, *242*, 675–683. [CrossRef] [PubMed]
31. Zhou, H.; Zhang, F.; Du, Z.; Liu, R. Forecasting PM2.5 using hybrid graph convolution-based model considering dynamic wind-field to offer the benefit of spatial interpretability. *Environ. Pollut.* **2021**, *273*, 116473. [CrossRef] [PubMed]
32. An, Y.; Xia, T.; You, R.; Lai, D.; Liu, J.; Chen, C. A reinforcement learning approach for control of window behavior to reduce indoor PM2.5 concentrations in naturally ventilated buildings. *Build. Environ.* **2021**, *200*, 107978. [CrossRef]
33. Kang, Z.; Qu, Z. Application of BP neural network optimized by genetic simulated annealing algorithm to prediction of air quality index in Lanzhou. In Proceedings of the 2017 2nd IEEE International Conference on Computational Intelligence and Applications (ICCIA), Beijing, China, 8–11 September 2017; pp. 155–160.
34. Li, M.; Wu, W.; Chen, B.; Guan, L.; Wu, Y. Water quality evaluation using back propagation artificial neural network based on self-adaptive particle swarm optimization algorithm and chaos theory. *Comput. Water Energy Environ. Eng.* **2017**, *6*, 229. [CrossRef]
35. Hu, X.; Belle, J.H.; Meng, X.; Wildani, A.; Waller, L.A.; Strickland, M.J.; Liu, Y. Estimating PM2.5 concentrations in the conterminous United States using the random forest approach. *Environ. Sci. Technol.* **2017**, *51*, 6936–6944. [CrossRef]